United International University
Department of Computer Science and Engineering
CSE 4811 / DS 4213: Natural Language Processing

Fall 2026

Practice Problems: Class 04

The neural language model

1. **Word vectors.**
   (a) True or false: one-hot vector encodings of words must be the length of the entire vocabulary, while learned embeddings are generally much smaller.
   (b) The input embedding table E and the output layer use separate parameters. The table is used only for lookup. One forward pass looks up IDs $[2, 5, 2]$. In the backward pass, the loss gradients with respect to the three returned vectors are $[2, 1]$, $[0, 3]$ and $[-1, 4]$, in that order. These are the only uses of the table. Write the two-component gradient of the loss for row 2 of E.

   UW *CSE 493G1 Deep Learning* **2024** Quiz 3, Q1.1 · Solutions

   Johns Hopkins *601.471 NLP: Self-supervised Models* **2026** Quiz 4, Q1(b) · Solutions

2. **Dense and one-hot vectors.** We have learned that dense word vectors learned from a large amount of text have many advantages over using sparse one-hot word vectors. Which of the following is NOT an advantage dense vectors have over sparse vectors?

   A. Models using dense word vectors generalize better to unseen words (words that never occur in the text the vectors are learned from) than those using sparse vectors.
   B. Models using dense word vectors generalize better to rare words than those using sparse vectors.
   C. Dense word vectors encode similarity between words while sparse vectors do not.
   D. Dense word vectors are easier to include as features in machine learning systems than sparse vectors.

   Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 3, Q1(e) · Solutions

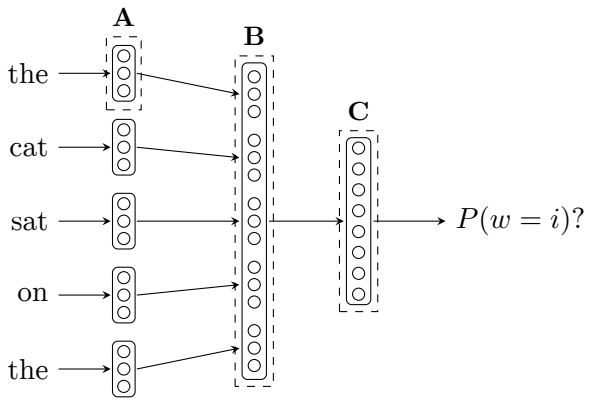
3. **One-hot inputs.** Consider the hidden layer of a neural language model that reads the two previous words,

$$h_t = \sigma\big(W\,[e'(w_{t-2});\, e'(w_{t-1})] + b\big),$$

   where $[\cdot\,;\,\cdot]$ joins two vectors into one. Let $e' \colon V \to \mathbb{R}^m$ be any (arbitrarily complex) word representation function. Let $\text{onehot}(w) \in \mathbb{R}^{|V|}$ be the one-hot representation of the word w: a vector of zeros with a 1 at the position of w.
   (a) Do we lose any representational power with neural language models if we restrict ourselves to simple one-hot representations of words?
   (b) Why would it still be advantageous to consider other, non-one-hot, representation functions e'?
   (c) Which words would you expect to get similar representations $e'(w)$?

   ETH Zurich *Large Language Models* **2024** Week 3 exercises, Q2 · Solutions

4. **Layers and sizes.** Answer the following questions according to the figure.



(a) Which language model algorithm is described by the computational graph shown in the figure?

A. N-gram language model
B. Fixed-window language model
C. Transformer with attention
D. Recurrent neural network (RNN)

(b) In the corresponding language model, given that the number of input tokens is 5, the embedding dimension of each individual token is d, and the hidden layer dimension is h, what are the dimensions of the vectors A, B and C shown in the figure?

(i) $\mathbb{R}^{5d}, \mathbb{R}^{5d}, \mathbb{R}^{5d}$ (ii) $\mathbb{R}^{d}, \mathbb{R}^{5d}, \mathbb{R}^{h}$ (iii) $\mathbb{R}^{d}, \mathbb{R}^{3d}, \mathbb{R}^{3d}$
(iv) $\mathbb{R}^{d}, \mathbb{R}^{h}, \mathbb{R}^{h}$ (v) $\mathbb{R}^{d}, \mathbb{R}^{5d}, \mathbb{R}^{5d}$ (vi) $\mathbb{R}^{d}, \mathbb{R}^{3d}, \mathbb{R}^{h}$

EPFL *CS-552 Modern Natural Language Processing* **2025** Make-up midterm, Q18 and Q20 · Solutions

5. **Counting parameters.** Consider a feedforward language model. In this model, the probability $P(w_i \mid w_{i-n+1}, \ldots, w_{i-1})$ is given by:

$$\begin{aligned} P(w_i \mid w_{i-n+1}, \ldots, w_{i-1}) &= \text{softmax}(\mathbf{U}\mathbf{h}_2 + \mathbf{b}_2) \\ \mathbf{h}_2 &= \tanh(\mathbf{W}\mathbf{h}_1 + \mathbf{b}_1) \\ \mathbf{h}_1 &= \text{concatenate}(\mathbf{C}\mathbf{w}_{i-n+1}, \ldots, \mathbf{C}\mathbf{w}_{i-1}) \\ \mathbf{w}_i &= \text{onehot}(w_i) \quad \text{for all } i \end{aligned}$$

In this notation, $\mathbf{U}$, $\mathbf{W}$ and $\mathbf{C}$ are matrices, while $\mathbf{b}_1$, $\mathbf{b}_2$, $\mathbf{h}_1$, $\mathbf{h}_2$ and $\mathbf{w}_{i-n+1}, \ldots, \mathbf{w}_{i-1}$ are vectors. The parameters of the model are $\mathbf{U}$, $\mathbf{W}$, $\mathbf{C}$, $\mathbf{b}_1$ and $\mathbf{b}_2$, while the remaining variables are intermediate layers computed by the network. Now consider the number of parameters required to represent this model. This number is determined by the size of the vocabulary $|V|$ (given to you by the data), the order n, and the dimension of the two hidden layers, $\mathbf{h}_1$ and $\mathbf{h}_2$, which we will denote d_1 and d_2, respectively (note that the dimension d_1 must be divisible by $n-1$, but you can ignore this detail in your calculations).

(a) How would you express the number of model parameters in terms of $|V|$, n, d_1 and d_2?

(b) An effective size for the hidden dimension of a neural NLP model is often in the hundreds. Let $|V| = 20{,}000$. For n from 2 to 5, how many parameters would your model have if $d_1/(n-1) = d_2 = 100$? What if $d_1/(n-1) = d_2 = 1000$?

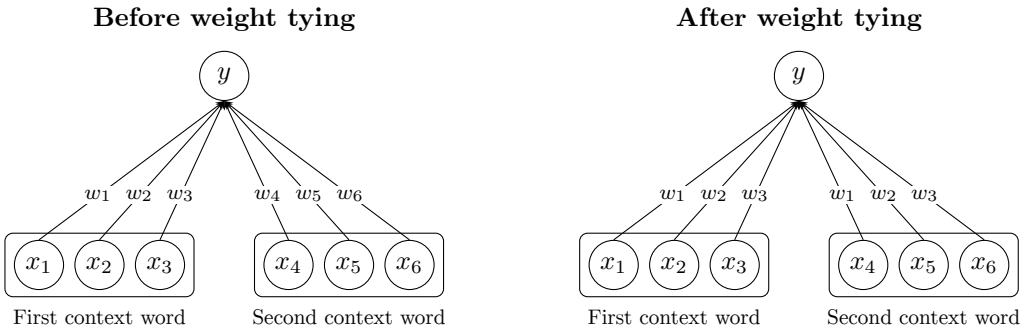
(c) What do you conclude about the relative memory efficiency of classic n-gram and feedforward neural language models? If you increased n even further, what would happen?

Edinburgh *Natural Language Understanding, Generation, and Machine Translation* **2025** Tutorial 1, Q2 · Solutions

6. **Tying weights.** Suppose that we have a vocabulary of 3 words, "a", "b" and "c", and we want to predict the next word in a sentence given the previous two words. For this network, we do not want to use feature vectors for words: we simply use the one-hot encoding, i.e. a 3-component vector with one entry being 1 and the other two entries being 0.

In the language models that we have seen so far, each of the context words has its own dedicated section of the network, so we would encode this problem with two 3-dimensional inputs. That makes for a total of 6 dimensions. For example, if the two preceding words (the "context" words) are "c" and "b", then the input would be $(0, 0, 1, 0, 1, 0)$. Clearly, the more context words we want to include, the more input units our network must have. More inputs mean more parameters, and thus increase the risk of overfitting. Here is a proposal to reduce the number of parameters in the model.

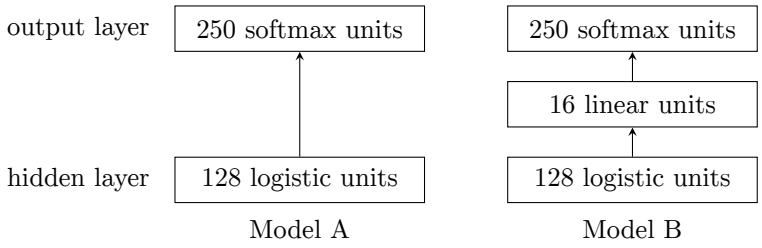
Consider a single neuron that is connected to this input, and call the weights that connect the input to this neuron w_1, w_2, w_3, w_4, w_5 and w_6. w_1 connects the neuron to the first input unit, w_2 connects it to the second input unit, and so on. Notice how for every neuron, we need as many weights as there are input dimensions (6 in our case), which will be the vocabulary size times the number of context words. A way to reduce the number of parameters is to tie certain weights together, so that they share a parameter. One possibility is to tie the weights coming from input units that correspond to the same word but at different context positions. In our example that would mean that $w_1 = w_4$, $w_2 = w_5$ and $w_3 = w_6$ (see the "after" diagram).



Explain the main weakness that this change creates.

Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2013** Final, B10

7. **A linear bottleneck.** Let us compare the following two models. Model A is a neural probabilistic language model whose hidden layer has 128 logistic units and whose output layer is a softmax over a vocabulary of 250 words. Model B is the same as Model A, with the following modification: in between the hidden layer (of size 128) and the output layer (of size 250), we insert a layer consisting of 16 linear units. The top layers of both models are shown in the figure. Describe one advantage of Model A and one advantage of Model B.



Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2015** Midterm, version 1, Q6 · Solutions

8. **Output word vectors.** I have an output matrix $U \in \mathbb{R}^{|V| \times d}$, with one row u_w for each word w, and a neural network that provides representations h of prefixes $w_{<t}$. One of my rows, u_1, is equal to $2u_2$. Another one of my rows, u_3, is equal to $-u_2$. Probabilities in the network are modeled as

$$P(\cdot \mid w_{<t}) = \text{softmax}(Uh).$$

What can we say about the relationships of the probabilities of words 1, 2 and 3 in this model? Specify which (all, or any subset) of words 1, 2 and 3 can be the maximum-probability word of the three for some prefix $w_{<t}$.

Columbia *COMS 4705 Natural Language Processing* **2026** Exam 1 preparation, Problem 5 · Solutions

9. **A simple language model.** Consider the following model. I have a vocabulary V, sequences $w_{1:T} \in V^*$, embeddings $E_w \in \mathbb{R}^d$ for every $w \in V$, and I make a probability distribution:

$$p(w \mid w_{<t}) = \exp\big(E_w^\top h_{w_{<t}}\big), \qquad h_{w_{<t}} = \frac{1}{t-1}\sum_{j=1}^{t-1} E_{w_j}, \qquad t \geq 2.$$

   (a) Let $w_{<t}$ be a prefix and $g \in V$ be some word such that $g \notin w_{<t}$, and compute

$$\nabla_{E_g}\big[-\log p(g \mid w_{<t})\big].$$

   (b) Unfortunately, this is not a language model. Describe what requirement of a language model it is missing, and minimally change it to solve this problem. (Note: we are not talking about a lack of expressivity in the computation of $h_{w_{<t}}$.)

Columbia *COMS 4705 Natural Language Processing* **2025** Exam 1, Problem 5 · Solutions

10. **Self-supervised learning.**

   (a) A training program takes the raw sentence "small birds build nests" and creates the input "small birds build" with target "nests". What makes this example self-supervised?

   A. The target is a human-written label describing the sentence's topic.
   B. The target is constructed automatically from the original text.
   C. The model has no prediction target.
   D. The model must predict every training example correctly.

   (b) Briefly explain a way in which the neural probabilistic language model was doing supervised learning, and a way in which it was doing unsupervised learning.

Johns Hopkins *601.471 NLP: Self-supervised Models* **2026** Quiz 1, Q1(a) · Solutions

Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2015** Midterm, version 2, Q6 · Solutions

11. **Softmax and cross-entropy.** Consider training a neural network for K-class classification using softmax activation and cross-entropy (CE) loss; for a language model, the K classes are the words of the vocabulary. For a given input x and its one-hot encoded label $y \in \{0,1\}^K$, the network outputs a vector of logits, the scores before the softmax, $z = (z_1, \ldots, z_K) \in \mathbb{R}^K$. Assume that the correct class is c ($y_c = 1$ and $y_i = 0$ for all $i \neq c$). Recall the following definitions:

$$\hat{y} = \text{softmax}(z) = \Big(\frac{\exp(z_1)}{Z}, \ldots, \frac{\exp(z_K)}{Z}\Big) \text{ where } Z = \sum_{i=1}^{K}\exp(z_i), \qquad L_{CE}(\hat{y}, y) = -\sum_{i=1}^{K} y_i \log \hat{y}_i.$$

In the following questions, you will express the cross-entropy loss in a different way.

   (a) LogSumExp (LSE), defined below, is an operation commonly encountered in deep learning:

$$\text{LSE}(x_1, \ldots, x_n) = \log\sum_{i=1}^{n}\exp(x_i) = \log\big(\exp(x_1) + \cdots + \exp(x_n)\big).$$

   Express the loss $L_{CE}(\hat{y}, y)$ in terms of the logits vector z and the LSE function. (Hint: $\hat{y} = \text{softmax}(z)$.)

   (b) Compute the partial derivative $\dfrac{\partial}{\partial z_j}\text{LSE}(z)$.

   (c) Compute the partial derivative $\dfrac{\partial}{\partial z_c} L_{CE}(\hat{y}, y)$ for the correct class c. (Hint: use part (a).)

   (d) Compute the partial derivative $\dfrac{\partial}{\partial z_j} L_{CE}(\hat{y}, y)$ for an incorrect class $j \neq c$.

   (e) Using the results of parts (c) and (d), express the gradient $\dfrac{\partial}{\partial z} L_{CE}(\hat{y}, y)$ using $\hat{y}$ and y.

Stanford *CS230 Deep Learning* **2018** Fall midterm, Q6(b) to (f) · Solutions

12. **Starting from zero.** Consider the following language model. I have a vocabulary V, sequences $w_{1:T} \in V^*$, embeddings $E_w \in \mathbb{R}^d$ for every $w \in V$, and I make a probability distribution:

$$p(w \mid w_{<t}) = \frac{\exp(E_w^\top h_{<t})}{\sum_{w' \in V} \exp(E_{w'}^\top h_{<t})}, \qquad h_{<t} = \frac{1}{t-1} \sum_{j=1}^{t-1} E_{w_j}, \qquad t \geq 2.$$

Suppose I have initialized all of my E_w to zero. Still, I am going to train all the E_w using gradient descent on the log-loss, as usual:

$$E \leftarrow E - \alpha \nabla_E \big(\text{the expected value of } -\log p(w_t \mid w_{<t}) \text{ over the training data}\big).$$

What is the value of this gradient? (You can write the value of a single gradient vector in $\mathbb{R}^d$ for an arbitrary word E_w.) Why does this make learning impossible? What is the probability distribution at initialization?

Columbia *COMS 4705 Natural Language Processing* **2025** Exam 1, Problem 3 · Solutions

13. **Gradient of the input.** A feedforward neural network language model (NNLM) can be used to train word vectors. This model tries to predict a word given the N words that precede it. To do so, we concatenate the word vectors of N previous words and send them through a single hidden layer of size H with a tanh nonlinearity and use a softmax layer to make a prediction of the current word. The size of the vocabulary is V. The model is trained using a cross entropy loss for the current word. Let the word vectors of the N previous words be $x_1, x_2, \ldots, x_N$, each a column vector of dimension D, and let y be the one-hot vector for the current word. The network is specified by the equations below:

$$x = [x_1; x_2; \ldots; x_N], \qquad h = \tanh(Wx + b), \qquad \hat{y} = \text{softmax}(Uh + d), \qquad J = -\sum_i y_i \log \hat{y}_i,$$

with $x \in \mathbb{R}^{ND}$, $W \in \mathbb{R}^{H \times ND}$, $b \in \mathbb{R}^H$, $h \in \mathbb{R}^H$, $U \in \mathbb{R}^{V \times H}$, $d \in \mathbb{R}^V$ and $\hat{y} \in \mathbb{R}^V$. What is the gradient of J with respect to x, $\partial J / \partial x$? Hint: $\tanh'(z) = 1 - \tanh(z)^2$.

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 3, Q3(c) · Solutions

14. **Rare words.** You are working with a colleague to build a neural language model for Polish, a morphologically rich language with many different inflectional forms for each lemma (dictionary form). For instance, *jabłko* (apple) can be inflected as: *jabłko*, *jabłka*, *jabłek*, *jabłku*, *jabłkom*, *jabłkiem*, *jabłkami*, *jabłkach*. As a consequence, many words appear very infrequently (if at all) in your training data. What kind of problem does this long tail of infrequent words create in neural language models? What are possible solutions? (Can you think of more than one?)

Edinburgh *Accelerated Natural Language Processing* **2025** Tutorial 2, Q2.1 · Solutions

15. **Counting and learning.**

(a) Consider how a neural language model with a softmax output layer compares with a classic n-gram language model. Typically, we use techniques like smoothing or backoff in conjunction with n-gram models. Does the problem these techniques solve arise in the neural model? Why or why not?

(b) Recall the neural language model architecture: the representations of the three previous words, rows of E, are joined into one vector e; the hidden units are $h = \sigma(We + b)$; and $\hat{y} = \text{softmax}(Uh)$ is the distribution for the next word. Why is it that the learned word representations for "professor" and "teach" are likely to be far apart? (It is not sufficient to say they are "semantically dissimilar"; your answer should mention how the representations are used in the prediction task.)

(c) Two prefixes are the same except for a name that appears earlier, and both end with the same four tokens, `the key belonged to`. A deterministic fixed-window language model sees only those four tokens. With the same model parameters used for both prefixes, can adding hidden layers alone make their next-token distributions differ? Answer yes or no, explain why in one sentence, and name one change to the input that would let the model tell the two prefixes apart.

Edinburgh *Natural Language Understanding, Generation, and Machine Translation* **2025** Tutorial 1, Q1(d) · Solutions

Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2017** Final, Q5 · Solutions

Johns Hopkins *601.471 NLP: Self-supervised Models* **2026** Quiz 4, Q1(c) · Solutions

16. **A stronger baseline.** Your colleague suggests that it would be helpful to implement a baseline model for a Polish language modeling task, so you can tell if the neural model is training effectively. Since multinomial logistic regression (MLR) is relatively straightforward to train, you choose that as your baseline. You both know that in general neural models work better than MLR for language modeling, so you expect that to be the case for your models as well. To implement your baseline, you use a multinomial logistic regression model that uses pre-trained embeddings of dimension d as input. Your neural model is a multilayer perceptron (MLP) that trains the embedding layer (again, using d-dimensional embeddings) as well as a single hidden layer with dimension d_h. Both models use a 3-word context window and the same vocabulary of size V.

    After training on a small dataset, your colleague reports that the MLR model performs better than the MLP. Can you hypothesize possible reasons why? (You should be able to think of more than one, especially since your colleague's statement is rather vague.) What are some questions you would ask your colleague and/or experiments you would run to help you figure out which hypothesis is most likely to be correct, and possible steps to improve the model?

    Edinburgh *Accelerated Natural Language Processing* **2025** Tutorial 2, Q3.4 · Solutions

17. **Part-of-speech tagging.** The next problem looks at how to apply neural models to a classical problem in NLP: part-of-speech (POS) tagging. POS tags are labels assigned to each word in a sentence to indicate its grammatical role in the sentence. For example:

i	1	2	3	4	5	6	7	8	9	10
x_i	Each	day	starts	with	one	or	two	lectures	by	researchers
y_i	DT	NN	VBZ	IN	CD	CC	CD	NNS	IN	NNS

    Common categories are Nouns (N), Pronouns (PRP), Verbs (VB), Adjectives (JJ), Adverbs (RB), Prepositions (IN), Determiners (DT), Conjunctions (CC) and Interjections (UH). There are also more fine-grained categories: the word *day* in the sentence is a singular noun (NN). However, the same word can have different POS tags, depending on the context.

    Given an input sentence $x = x_1 \dots x_{|x|}$, we want to predict the corresponding tag sequence $y = y_1 \dots y_{|x|}$. Let x_i denote the ith word of x, y_i denote the ith tag of y, and $|x|$ denote the length of x. Note that $|y| = |x|$. We have access to many training examples like this, and our goal is to model the conditional probability of the tag sequence given the sentence, that is, $P(y \mid x)$. There are many possible choices here. To simplify the problem, let us assume that each element of y is conditionally independent of the others. That is, we want to model:

$$P(y \mid x) = \prod_{i=1}^{|y|} P(y_i \mid x).$$

    Design a feedforward neural network to model $P(y_i \mid x)$: clearly define the probability being computed and how it is estimated, and specify the input and output vocabulary, as well as the objective function used for training. Identify any independence assumptions you make. Draw a diagram that illustrates how the model computes probabilities for the tag of the word "with": what is the input, and how is the output distribution computed from the input? Write out the basic equations of the model, and explain your choices.

    Edinburgh *Foundations of Natural Language Processing* **2025** Tutorial 3, Q3.1 · Solutions

Word vectors

18. **The distributional hypothesis.** Briefly describe what is meant by the distributional hypothesis and how it motivates the training of dense word vectors.

    Macalester *COMP 394 Natural Language Processing* **2024** Practice exam 2, Word embeddings (c) · Solutions

19. **Similar embeddings.** Suppose we have the embedding for a word a, and we find another word b that has an embedding with a very high cosine similarity to the embedding of a. Which of the following is most likely to be true?

    A. Words a and b are synonyms.
    B. The embeddings for a and b are only alike due to randomness in the vector space.
    C. Words a and b appear in a lot of the same contexts.
    D. Words a and b can be used interchangeably without affecting the meaning of sentences.

    Stanford *CS124 From Languages to Information* **2019** Practice final, Q18 · Solutions

20. **Antonyms.**

    (a) True or false: antonyms, such as "hot" and "cold", have very different context words.

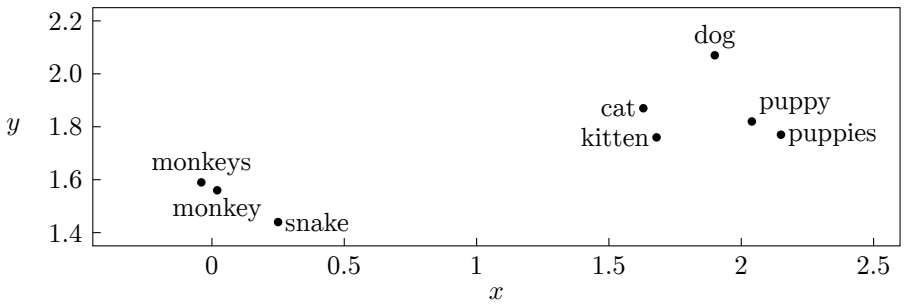
    (b) Embeddings help model semantics, because words with similar meanings often have similar embeddings. But they are known to fail in modeling one aspect of semantics: the difference between antonyms (words with opposite meaning) and synonyms (words with the same meaning). For example, the embedding for *love* is similar to the embeddings for *cherish* and *adore*, but also to the embedding for *hate*. Explain why a word's antonyms, like its synonyms, often lie near it in the embedding space.

    EPFL *CS-433 Machine Learning* **2022** Final, Q35 · Solutions

    Edinburgh *Accelerated Natural Language Processing* **2023** Tutorial 4, Problem 1.3(a) · Solutions

21. **A surprising neighbor.** Consider the following two-dimensional word vectors that encode animals. They are projections of 300-dimensional word vectors that were trained using data from all over the web.

	dog	puppy	puppies	cat	kitten	snake	monkey	monkeys
x	1.9	2.04	2.15	1.63	1.68	0.25	0.02	−0.04
y	2.07	1.82	1.77	1.87	1.76	1.44	1.56	1.59



    Visually inspect the word vectors. Distances in the vector space capture word similarities (in a much simplified way, since only two dimensions are used). Why is snake, a reptile, closer to monkey, a mammal, than to the other animals, which are also mammals? Please refer to the distributional hypothesis and the way word vectors are constructed in your answer.

    Edinburgh *Foundations of Natural Language Processing* **2024** Tutorial 5, Q2.1 · Solutions

22. **Skip-gram and CBOW.**

    (a) Select the answer that includes all the skip-gram (word, context) training pairs for the sentence *the cat ran away*, for a window size ±2 from the target.

    A. [the, cat], [the, ran], [cat, ran], [cat, away], [ran, away]

B. [the], [cat], [ran], [away]
C. [the, cat], [the, ran], [cat, the], [cat, ran], [cat, away], [ran, the], [ran, cat], [ran, away], [away, cat], [away, ran]
D. [the, ran], [ran, cat], [cat, away]
E. [the, cat ran], [cat, the], [cat, ran away], [ran, the cat], [ran, away], [away, cat ran]

(b) In one or two sentences, briefly explain the difference between the skip-gram and continuous bag-of-words (CBOW) methods.

UT Austin *CS388 Natural Language Processing* **2022** Midterm, Q1(1) and Q2(a) · Solutions

23. **CBOW and the NNLM.** A feedforward neural network language model (NNLM) can be used as another architecture for training word vectors. This model tries to predict a word given the N words that precede it. To do so, we concatenate the word vectors of N previous words and send them through a single hidden layer of size H with a tanh nonlinearity and use a softmax layer to make a prediction of the current word. The size of the vocabulary is V. The model is trained using a cross entropy loss for the current word. State two important differences between the NNLM and the CBOW model. Explain how each might affect the word vectors learned.

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 3, Q3(a) · Solutions

24. **Skip-gram probabilities.** Suppose you have skip-gram word embeddings for three words 1, 2, and 3 (word vectors w and context vectors c):

$$w_1 = (0,0), \quad w_2 = (1,1), \quad w_3 = (-1,1), \qquad c_1 = (0,0), \quad c_2 = (0,1), \quad c_3 = (1,0).$$

Assume for this question that e (the mathematical constant) is equal to 3.

(a) What is the distribution over context words for w_1?

(b) What is the distribution over context words for w_2?

UT Austin *CS371N Natural Language Processing* **2023** Midterm, Q3(a) and (b) · Solutions

25. **Word and context vectors.**

(a) What are two ways practitioners deal with having two different sets of word vectors, W and C, at the end of training word2vec?

(b) Alice and Bob have each used the word2vec algorithm to obtain word embeddings for the same vocabulary of words V. In particular, Alice has obtained "context" vectors c_i^A and "center" vectors w_i^A for every word $i \in V$, and Bob has obtained "context" vectors c_i^B and "center" vectors w_i^B for every $i \in V$. Suppose that, for every pair of words $i, j \in V$, the inner product is the same in Alice's and Bob's models: $c_i^A \cdot w_j^A = c_i^B \cdot w_j^B$. Does it follow that, for every word $i \in V$, $w_i^A = w_i^B$? Why or why not?

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 2, Q2.2 · Solutions

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Midterm, Q3(d) · Solutions

26. **The cost of the softmax.** A feedforward neural network language model (NNLM) can be used as another architecture for training word vectors. This model tries to predict a word given the N words that precede it. To do so, we concatenate the word vectors of N previous words and send them through a single hidden layer of size H with a tanh nonlinearity and use a softmax layer to make a prediction of the current word. The size of the vocabulary is V. The model is trained using a cross entropy loss for the current word. Let the word vectors of the N previous words be $x_1, x_2, \ldots, x_N$, each a column vector of dimension D, and let y be the one-hot vector for the current word. The network is specified by the equations below:

$$x = [x_1; x_2; \ldots; x_N], \qquad h = \tanh(Wx + b), \qquad \hat{y} = \text{softmax}(Uh + d), \qquad J = -\sum_i y_i \log \hat{y}_i,$$

with $x \in \mathbb{R}^{ND}$, $W \in \mathbb{R}^{H \times ND}$, $b \in \mathbb{R}^H$, $h \in \mathbb{R}^H$, $U \in \mathbb{R}^{V \times H}$, $d \in \mathbb{R}^V$ and $\hat{y} \in \mathbb{R}^V$.

(a) What is the complexity of forward propagation in an NNLM for a single training example?

(b) Describe at least one way to change the model that would reduce this complexity.

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 3, Q3(b) · Solutions

27. **Skip-gram for bigrams.** Suppose you want to use the skip-gram model to learn bigram embeddings instead of word embeddings. Recall the basic skip-gram model:

$$P(\text{context} = y \mid \text{word} = x) = \frac{\exp(c_y \cdot w_x)}{\sum_{y' \in \text{vocab}} \exp(c_{y'} \cdot w_x)}.$$

You are going to learn a bigram vector for every bigram. That is, for a sentence like *the cat saw*, you will form the (word, context) pairs (*the cat*, *saw*) and (*cat saw*, *the*), where the first item of each pair is a bigram and the second a unigram. Note that contexts are still unigrams.

(a) With w and c denoting the word and context vectors respectively, and b denoting a bigram, write out the model for skip-gram with word bigrams and context unigrams:

$$P(\text{context} = y \mid \text{word} = b) = ?$$

(b) What is the big-O runtime of computing the probability for a single bigram-context pair? Express this answer in terms of $|V|$, the vocabulary size, $|B|$, the number of bigrams attested in the data, and d, the dimension of the vectors. Hint: taking the dot product of two d-length vectors is an $O(d)$ operation.

(c) Assume that making a single gradient step on a single example takes the same time as what you reported in part (b). What is the big-O runtime of model training for M (bigram, context) examples for N epochs? Express this in terms of your answer to part (b), which you can denote as E. (You may also use the other quantities defined in the part (b) question as needed.)

(d) How many parameters are in the model? Express this in terms of the quantities in part (b). Your answer to this part should be exact.

UT Austin *CS371N Natural Language Processing* **2022** Midterm, Q5 · Solutions

28. **A Siamese network.** Siamese neural nets have an interesting architecture: the same parameters and functions are used to evaluate two inputs. As one might expect, Siamese nets are useful to train similarity metrics, evaluations of how "close" inputs are. These nets have been applied to facial recognition tasks with a good deal of success, but in this example, we will see how to train Siamese nets to learn a distance metric for word vectors. One might imagine training a net to map word vectors across languages, discover synonyms or antonyms, etc.

Here is one such model to evaluate how similar two input words are using Euclidean distance. There are two input word vectors $x_1, x_2 \in \mathbb{R}^n$, shared parameters $W \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$, and a single hidden layer associated with each input:

$$h_1 = \sigma(Wx_1 + b), \qquad h_2 = \sigma(Wx_2 + b).$$

We evaluate the distance between the two activations h_1, h_2 using Euclidean distance as our similarity metric. The model objective J is

$$J = \tfrac{1}{2}\|h_1 - h_2\|^2.$$

How does this model compare to the skip-gram model? Give one qualitative similarity or one difference.

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 1, Q1.4 · Solutions

29. **Solving analogies.** Consider the following two-dimensional word vectors, which are projections of 300-dimensional word vectors trained using data from all over the web:

	dog	puppy	puppies	cat	kitten	snake	monkey	monkeys
x	1.9	2.04	2.15	1.63	1.68	0.25	0.02	−0.04
y	2.07	1.82	1.77	1.87	1.76	1.44	1.56	1.59

Word vectors can capture relationships between words. An example of such a relationship is an analogy, such as Edinburgh is to Scotland as Canberra is to ...? When we have word vectors, we

can use the parallelogram method to solve analogies: by subtracting $\overrightarrow{\text{edinburgh}}$ from $\overrightarrow{\text{scotland}}$ and adding $\overrightarrow{\text{canberra}}$. You would pick the word for which the word vector has the smallest distance to the resulting vector. For the analogy a:b::a*:b* (a is to b as a* is to b*), the procedure is

$$\vec{b}^* = \underset{\vec{x}}{\text{argmin}}\ \text{distance}\big(\vec{x},\ \vec{b} - \vec{a} + \vec{a}^*\big),$$

where $\vec{x}$ ranges over the words other than the three given words. Using Euclidean distances, compute $\vec{b}^*$ for:

- $\vec{a} = \overrightarrow{\text{dog}}$, $\vec{b} = \overrightarrow{\text{puppy}}$ and $\vec{a}^* = \overrightarrow{\text{cat}}$;
- $\vec{a} = \overrightarrow{\text{puppy}}$, $\vec{b} = \overrightarrow{\text{puppies}}$ and $\vec{a}^* = \overrightarrow{\text{monkey}}$.

Edinburgh *Foundations of Natural Language Processing* **2024** Tutorial 5, Q2.2 · Solutions

30. **Claims about analogies.** Pretrained embedding spaces often enable analogy arithmetic. The following statements are labeled A–E. Which option correctly identifies the set of true statements?

    A. `king` − `man` + `woman` ≈ `queen`.
    B. `paris` − `france` is approximately the "capital" vector in embedding space.
    C. `doctor` − `man` + `woman` is necessarily unbiased.
    D. Analogy arithmetic reflects approximate non-linear relational structure in embedding spaces.
    E. Strong analogy performance implies the embeddings are unbiased.

    Options:

    1. B only
    2. B and D
    3. A, B, and D
    4. A, C, and E
    5. None of the other options is correct.
    6. A only
    7. C, D, and E
    8. A and B

    EPFL *CS-552 Modern Natural Language Processing* **2026** Midterm, Q18 · Solutions

31. **Evaluating word vectors.** Give two examples of how we can evaluate word vectors. For each example, please indicate whether it is intrinsic or extrinsic. (An intrinsic evaluation tests the vectors themselves on a small task about words; an extrinsic evaluation tests them as part of a system for a real application.)

    Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Midterm, Q3(b) · Solutions

32. **Supervised or self-supervised.** Noah, a student in this course, trains embeddings with a supervised objective: predicting the sentiment label of each review. Lucie trains embeddings with self-supervised learning. The following statements are labeled A–D. Which option correctly identifies the set of true statements?

    A. Noah must acquire a lot of labeled data.
    B. The embeddings Noah gets will reliably transfer to part-of-speech (POS) tagging, the marking of each word as a noun, a verb and so on.
    C. Lucie's embeddings are guaranteed better than Noah's for every downstream task.
    D. Lucie's approach is more aligned with distributional semantics ("you know a word by the company it keeps").

    Options:

    1. A only
    2. B and C
    3. A, B, and D
    4. None of the other options is correct.
    5. A and D
    6. B, C, and D
    7. A and C
    8. D only

    EPFL *CS-552 Modern Natural Language Processing* **2026** Midterm, Q21 · Solutions

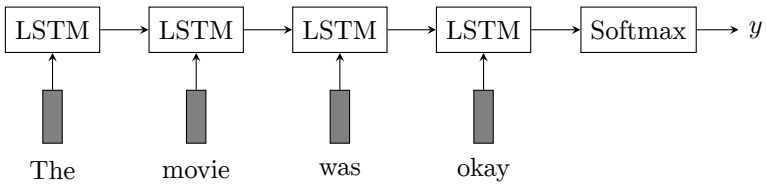
33. **Averaged word vectors.**

    (a) What is one advantage of using averaged word embeddings, instead of bag-of-words, for a document classifier?

(b) What is one disadvantage of using averaged word embeddings, instead of bag-of-words, for a document classifier?

UMass Amherst *CS485 Natural Language Processing* **2024** Practice midterm, Q4.3 and 4.4

34. **Fine-tuning word vectors.** A popular model used for sentiment classification is an LSTM model:



This model inputs word vectors to the LSTM model at each time step and uses the last hidden state vector to predict the sentiment label (y). Assume that the word vectors of the LSTM model were initialized using word2vec. If we chose to update our word vectors when training the LSTM model on sentiment classification data, how would these word vectors differ from ones not updated during training? Explain with an example.

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 3, Q2(d)ii · Solutions

35. **Retraining on small data.** Although pre-trained word vectors work very well in many practical downstream tasks, in some settings it is best to continue to learn (i.e. "retrain") the word vectors as parameters of our neural network. Explain why retraining the word vectors may hurt our model if our dataset for the specific task is too small.

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Midterm, Q3(a) · Solutions

36. **A word sense classifier.** Danyi and Natalya are using a large corpus of historical documents to study how words start to be used as slang terms, and how the meanings change over time. For example, the word *hip* originally only referred to a part of the body, but then acquired a new meaning as a slang term meaning *trendy*.

To help analyze their documents, Danyi and Natalya want to build a word sense disambiguation system: that is, a classifier that can tell them which sense (meaning) of a word is being used in a particular example.

They have just learned about word2vec. Natalya thinks that word embeddings will be useful for building this system. She initially comes up with the following model, where $v(w)$ is the word2vec embedding for some word w in the vocabulary; $S(w)$ is the set of possible senses for w; s_i is a particular sense; and θ are model parameters:

$$P(s_i \mid w, \theta) = \frac{\exp(\theta \cdot v(w))}{\sum_{s_j \in S(w)} \exp(\theta \cdot v(w))}.$$

(a) What type of model is this? (That is, what is it called?)

(b) When Danyi looks at Natalya's model, she realizes that, as stated, it is not going to work for the task of word sense disambiguation. Explain why.

(c) How would you adapt the model so that it can (in principle) solve this task, still using only word2vec embeddings?

(d) Suppose Danyi and Natalya decide to use your model for their task. What sort of training data will they need? How feasible do you think it would be for them to do a large-scale study of many different words, using this approach?

Edinburgh *Accelerated Natural Language Processing* **2025** Tutorial 2, Problem 1 · Solutions

37. **Items bought together.** Word2vec represents a family of embedding algorithms that are commonly used in many applications. Suppose in a recommender system for online shopping, we have information about co-purchase records for items $x_1, x_2, \ldots, x_n$ (for example, item x_i is commonly bought together with item x_j). Explain how you would use ideas similar to word2vec to recommend similar items to users who have shown interest in any one of the items.

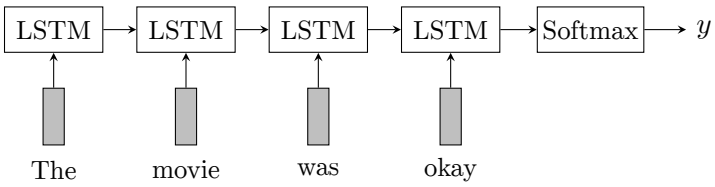
Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Midterm, Q2(c) · Solutions

Recurrent networks

38. **Long-distance agreement.** We briefly discussed how n-gram models can fail to capture phenomena like agreement. For example, a 4-gram model cannot, in principle, reliably predict whether the verb that comes after *the dog near the gardens* is singular or plural. You can construct such an example for an n-gram model for any choice of n, including a fixed-window neural language model. Briefly explain why you cannot construct such an example for an RNN model, at least in principle.

    Macalester *COMP 394 Natural Language Processing* **2024** Practice exam 2, Modern NN Architectures (a) · Solutions

39. **Word order.** A popular model used for sentiment classification is an LSTM model:



    This model inputs word vectors to the LSTM model at each time step and uses the last hidden state vector to predict the sentiment label (y). A simpler "bag-of-vectors" model for sentiment classification uses the average of all the word vectors in a sentence to predict the sentiment label. Name at least one benefit of the LSTM model over the bag-of-vectors model.

    Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 3, Q2(d)(i) · Solutions

40. **Parameters of an RNN.**

    (a) True or false: The number of parameters in an RNN increases linearly with the sequence length.

    (b) A feedforward neural language model of order n predicts each word from the $n-1$ words before it. It joins the vectors of these words, m numbers each, into one layer of $(n-1)m$ numbers. This layer feeds a hidden layer of d units, and then a softmax over a vocabulary of $|V|$ words. Counting the word table, the weights and the biases, it has

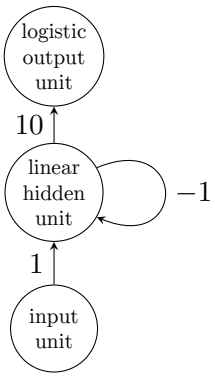
$$(1+m+d)\,|V| + (n-1)\,m\,d + d$$

    parameters. How would you expect the number of parameters in an RNN model to scale with n and $|V|$?

    UW *CSE 493G1 Deep Learning* **2026** Exam 2, Q1.4 · Solutions

    Edinburgh *Natural Language Understanding, Generation, and Machine Translation* **2025** Tutorial 1, Q2(d) · Solutions

41. **A small RNN.** Determine what the following recurrent network computes. More precisely, determine the function computed by the output unit at the final time step; the other outputs are not important. All of the biases are 0, and the hidden unit starts at 0. You may assume the inputs are integer valued and the length of the input sequence is even.



    Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2017** Final, Q10 · Solutions

42. **Choosing RNN weights.** Ronnie makes a simple RNN with state dimension 1 and a *step* function as its activation, so that

$$s_t = \text{step}(w_1 x_t + w_2 s_{t-1} + b),$$

where step$(z) = 1$ if $z > 0.0$ and equals 0 otherwise, and where the output is $y_t = s_t$.

(a) Assuming $s_0 = 0$, what values of w_1, w_2 and b would generate output sequence $[0, 0, 0, 1, 1, 1, 1]$ given input sequence $[0, 0, 0, 1, 0, 1, 0]$?

(b) Now Ronnie wants to make their machine generate output sequence $[1, 1, 1, 0, 0, 0, 1, 1]$ given input sequence $[0, 0, 0, 1, 0, 0, 1, 0]$. Assuming $s_0 = 1$, provide the desired s_t value for each possible combination of x_t and s_{t-1} values, for this example sequence.

x_t	s_{t-1}	s_t
0	0	
0	1	
1	0	
1	1	

(c) Rennie thinks this is not possible using Ronnie's architecture. Rennie makes an argument based on the relationships in the table above. Is Rennie right? If it is not possible, explain why and describe a change to this architecture that can implement this mapping. If it is possible, provide parameters w_1, w_2 and b that implement this mapping.

MIT *6.036 Introduction to Machine Learning* **2021** Final, Q11 · Solutions

43. **Training and generation.** Suppose you wish to use a recurrent neural network language model (RNNLM) to compute the log-likelihood of a sequence of words $w_1, w_2, \ldots, w_T$. At each timestep the RNNLM reads one token and then computes its output softmax from its new hidden state.

(a) True or false: The first token fed into the RNNLM is w_1.

(b) Fill in the blank: The output softmax at timestep t represents _____.

A. the conditional probability distribution over w_t given w_{t-1}
B. the conditional probability distribution over w_t given $w_{t-1}, w_{t-2}, \ldots, w_1$
C. the marginal probability distribution over w_t
D. the marginal probability distribution over w_{t-1}
E. None of the above

(c) Now suppose we are training an RNN language model using teacher forcing. What are the inputs to the network at training time?

(d) What are the inputs to the network at test time, when it generates text?

CMU *10-418 Machine Learning for Structured Data* **2022** Practice problems 1, Q2.1(b) and (c) · Solutions

Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2018** Final, Q5 · Solutions

44. **Paths through time.** Consider the equations for a simple recurrent model mapping an input $x^{(1)} \ldots x^{(n)}$ to a label y.

- Inputs: a sequence $x^{(1)} \ldots x^{(n)}$ where each $x^{(t)} \in \mathbb{R}^d$; a label $y \in \{1 \ldots K\}$; an integer m defining the size of the hidden dimension.
- Parameters: $W^{hh} \in \mathbb{R}^{m \times m}$, $W^{hx} \in \mathbb{R}^{m \times d}$, $b^h \in \mathbb{R}^m$, $h^{(0)} \in \mathbb{R}^m$, $V \in \mathbb{R}^{K \times m}$, $\gamma \in \mathbb{R}^K$; a transfer function (that is, an activation function) $g \colon \mathbb{R}^m \to \mathbb{R}^m$.
- Computational graph: for $t = 1 \ldots n$, $h^{(t)} = g(W^{hx} x^{(t)} + W^{hh} h^{(t-1)} + b^h)$; then $l = V h^{(n)} + \gamma$, $q = \text{LS}(l)$ and $o = -q_y$. Here LS is the log-softmax, $q_k = \log \text{softmax}(l)_k$, so o is the cross-entropy loss.

(a) Draw the computational graph for the above equations with $n = 3$. There are three directed paths in the computational graph from W^{hx} to o. One of them is $W^{hx} \to h^{(3)} \to l \to q \to o$. What are the other two directed paths?

(b) For each pair of variables (a, b) with a directed arc from a to b in the computational graph, define $D(a \to b)$ to be the Jacobian associated with the edge from a to b, as calculated in the backpropagation algorithm. (When a is a matrix, such as W^{hx}, its entries are treated as one long vector.) For example, $D(q \to o)$ is the Jacobian on the edge from q to o. For convenience, define the matrix $A = D(q \to o) \times D(l \to q) \times D(h^{(3)} \to l)$ and in addition define matrices $B_2 = D(h^{(2)} \to h^{(3)})$ and $B_1 = D(h^{(1)} \to h^{(2)})$. Now write down the expression for $\partial o / \partial W^{hx}$

using the fact that we can sum over all paths from W^{hx} to o, taking a product of Jacobians along each path.

Columbia *COMS W4705 Natural Language Processing* **2018**
Questions on recurrent networks, Q1(a) and (b) · Solutions

45. **Sigmoid and tanh.**

(a) In a vanilla RNN, the hidden state is computed as $h_t = \sigma(W_{hx}x_t + W_{hh}h_{t-1} + b_h)$. When computing the gradient $\partial h_t/\partial h_{t-T}$ during backpropagation through time, the result is a product of T terms, each containing W_{hh} and the derivative of the activation function. Which of the following most accurately explains why using a sigmoid activation makes the vanishing gradient problem particularly severe?

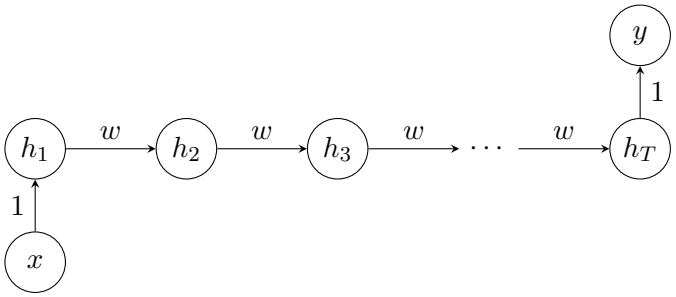
A. The sigmoid derivative is always exactly 0.5, which causes the gradient to halve at every time step.
B. The sigmoid function is not differentiable at certain points, causing gradient computation to fail for long sequences.
C. The sigmoid function saturates at 0 and 1, meaning its output is always less than 1, which directly causes the gradient to shrink.
D. The sigmoid derivative has a maximum value of 0.25, so each multiplicative factor in the chain is at most 0.25, causing exponential decay over T steps.

(b) True or false: The backpropagated gradient through a tanh non-linearity is always smaller than or equal in magnitude to the upstream gradient. (Recall: if $z = \tanh(x)$ then $\partial z/\partial x = 1 - z^2$.)

EPFL *CS-552 Modern Natural Language Processing* **2026** Midterm, Q1 · Solutions

Stanford *CS230 Deep Learning* **2019** Midterm, Q1(f) · Solutions

46. **Gradients through time.** Consider the following RNN, which has a scalar input at the first time step, makes a scalar prediction at the last time step, and uses a shifted logistic activation function: $\phi(z) = \sigma(z) - 0.5$.



For $t < T$, backpropagation gives $\partial L/\partial h_t = w\,\sigma'(z_{t+1})\,\partial L/\partial h_{t+1}$, where L is the loss computed from the prediction y and z_t is the input to the activation function at time t.

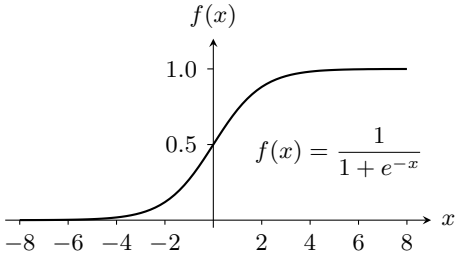
(a) Suppose the input to the network is $x = 0$, and assume $w > 0$. Notice that $h_t = 0$ for all t. Based on the formula above, determine the value α such that if $w < \alpha$, the gradient vanishes, while if $w > \alpha$, the gradient explodes. You may use the fact that $\sigma'(0) = 1/4$.

(b) True or false: clipping the gradient (cutting off at a threshold) will solve the exploding gradients problem. Briefly justify your answer.

Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2017** Final, Q12 · Solutions

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 2, Q4.1(d) · Solutions

47. **Saturation and ReLU.** Consider the sigmoid activation function:



(a) What would the gradient of the sigmoid be for a very large input?

(b) Is this a potential problem when training an RNN? Explain.

(c) How does ReLU activation ($\text{ReLU}(z) = \max(0, z)$) compare to sigmoid activation? Does it solve the problem above completely?

(d) Which of the following is true about the vanishing gradient problem? Select all that apply.

(i) Tanh is usually preferred over sigmoid because it does not suffer from vanishing gradients.

(ii) Vanishing gradient causes deeper layers (near the output) to learn more slowly than earlier layers (near the input).

(iii) Leaky ReLU is less likely to suffer from vanishing gradients than sigmoid.

(iv) Xavier initialization can help prevent the vanishing gradient problem.

(v) None of the above.

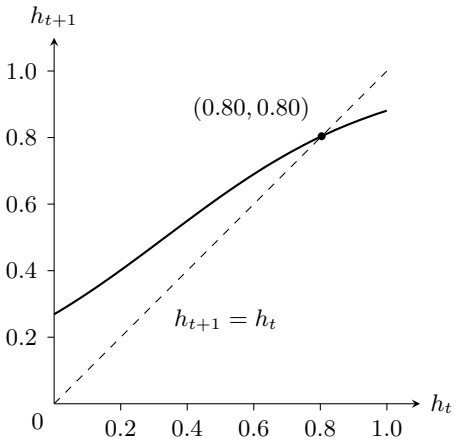
CMU *10-423 Generative AI* **2025** Practice exam, Q1.6 · Solutions

Stanford *CS230 Deep Learning* **2019** Midterm, Q1(e) · Solutions

48. **A sigmoid recurrence.** Recall that a recurrent neural network (RNN) takes in an input vector x_t and a state vector h_{t-1} and returns a new state vector h_t and an output vector y_t:

$$h_t = f(w_1 x_t + w_2 h_{t-1} + b_2), \qquad y_t = g(w_3 h_t + b_3),$$

where f and g are activation functions. Here x_t, h_{t-1}, h_t and y_t are all scalars. Suppose that f is a sigmoid unit and that the input is always 0. The following diagram shows how the next state h_{t+1} changes with the current state h_t when $w_2 = 3$ and $b_2 = -1$, i.e. $h_{t+1} = \sigma(3h_t - 1)$. The diagram also shows that $h_{t+1} = h_t$ when $h_t = 0.80$.



(a) Would h_{t+1} be greater than or less than h_t if $h_t = 0.5$?

(b) Would h_{t+1} be greater than or less than h_t if $h_t = 0.9$?

(c) What happens to h_t as we take increasingly longer sequences, i.e. as $t \to \infty$?

(d) For what range of initial values h_0 would this network experience exploding gradients for long sequences?

(e) For what range of initial values h_0 would this network experience vanishing gradients for long sequences?

(f) ReLUs are known to be good at preserving gradients in deep neural networks. Would it be wise to replace the sigmoid activation above with a ReLU activation unit? Explain.

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 3, Q5(b) · Solutions

49. **LSTM gates.** Here are the defining equations for an LSTM cell.

$$i_t = \sigma\big(W^{(i)}x_t + U^{(i)}h_{t-1}\big), \qquad f_t = \sigma\big(W^{(f)}x_t + U^{(f)}h_{t-1}\big), \quad o_t = \sigma\big(W^{(o)}x_t + U^{(o)}h_{t-1}\big),$$
$$\tilde{c}_t = \tanh\big(W^{(c)}x_t + U^{(c)}h_{t-1}\big), \quad c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t, \qquad h_t = o_t \circ \tanh(c_t).$$

Recall that $\circ$ denotes element-wise multiplication and that σ denotes the sigmoid function. True or false? Briefly justify each answer.

(a) If x_t is the 0 vector, then $h_t = h_{t-1}$.

(b) If f_t is very small or zero, then error will not be back-propagated to earlier time steps.

(c) The entries of f_t, i_t, o_t are non-negative.

(d) f_t, i_t, o_t can be viewed as probability distributions (that is, their entries are non-negative and their entries sum to 1).

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 2, Q4.2 · Solutions

50. **The LSTM memory cell.** Recall the LSTM architecture: the memory cell is updated by

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t.$$

The forget gate f_t and the input gate i_t are computed with the sigmoid, the new content $\tilde{c}_t$ with tanh, and $\odot$ is element-wise multiplication.

(a) Suppose you want the memory cell to sum its inputs over time, that is, the new contents $\tilde{c}_1, \tilde{c}_2, \ldots$ written to it. What values should the input gate and forget gate take? You do not need to justify your answer.

(b) Karpathy et al. (2015) showed that individual dimensions of the LSTM cell state c can learn to track semantic features such as code indentation depth and whether text is inside quotation marks. Which of the following statements is true about how LSTMs track long-range features?

A. The LSTM uses a softmax over cell dimensions, forcing exactly one dimension to be active at a time, which creates a natural specialization.

B. The element-wise (Hadamard) multiplication in $c_t = i_t \odot \tilde{c}_t + f_t \odot c_{t-1}$ allows each dimension of the cell state to be independently gated.

C. The LSTM architecture includes an explicit feature assignment mechanism that maps semantic concepts to specific cell dimensions during initialization.

D. The cell state is constrained to be binary (0 or 1), so each dimension naturally represents a boolean feature.

Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2017** Final, Q14 · Solutions

EPFL *CS-552 Modern Natural Language Processing* **2026** Midterm, Q2 · Solutions

51. **GRU gates.** Gated Recurrent Units (GRUs) are an extension of RNNs. GRUs can adaptively reset or update their memory of previous states. The forward pass of a GRU is given by

$$z_t = \sigma(W_z x_t + U_z h_{t-1}), \qquad r_t = \sigma(W_r x_t + U_r h_{t-1}),$$
$$\tilde{h}_t = \tanh(W x_t + r_t \circ U h_{t-1}), \quad h_t = (1 - z_t) \circ h_{t-1} + z_t \circ \tilde{h}_t,$$

where $\circ$ is element-wise multiplication.

(a) Explain the role of the z_t and r_t vectors in these updates.

(b) True or false: If the update gate z_t is close to 0, the net does not update its state significantly.

(c) True or false: If the update gate z_t is close to 1 and the reset gate r_t is close to 0, the net remembers the past state very well.

Stanford *CS224N Natural Language Processing with Deep Learning* **2018**
Practice midterm 1, Q3.2 and Q3.3 · Solutions

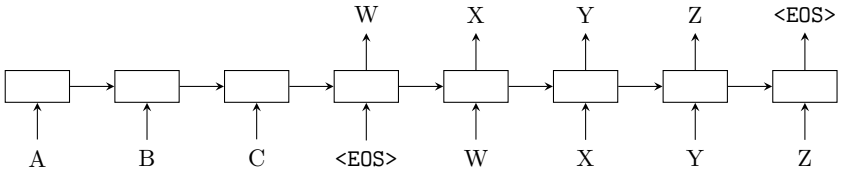
Columbia *COMS W4705 Natural Language Processing* **2018** Questions on recurrent networks, Q3(a) · Solutions

52. **Gradients in an LSTM.** Select all that apply:
    A. One motivation of LSTMs is to reduce the vanishing gradient problem seen with RNNs.
    B. It is not possible for LSTMs to experience the vanishing gradient problem.
    C. It is possible for LSTMs to experience the vanishing gradient problem.
    D. It is not possible for LSTMs to experience the exploding gradient problem.
    E. It is possible for LSTMs to experience the exploding gradient problem.

CMU *10-423 Generative AI* **2025** Practice exam, Q1.4 · Solutions

53. **Translation with an RNN.**

(a) We can apply a recurrent net to machine translation by feeding it an English sentence, and then having it generate the French sentence the same way an RNN language model generates text. This setup is shown in the figure. Would it work to use the neural probabilistic language model of Bengio et al. (2003) in the same way? Why or why not?



(b) Briefly explain one flaw of encoder-decoder architectures for machine translation.

Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2015** Midterm (version 1), Q8 · Solutions

Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2018** Final, Q9 · Solutions

54. **Encoder and decoder.** A recurrent seq2seq model feeds an input sequence $[a_1, a_2, \ldots]$ to a unidirectional LSTM encoder to obtain a hidden state sequence $[b_1, b_2, \ldots]$. Here each b_t includes the memory cell of the LSTM. The last hidden state b_{LAST} is fed in as the zeroth hidden state of an RNNLM decoder (i.e. c_0). The RNNLM decoder produces the hidden sequence $[c_1, c_2, \ldots]$, a sequence of output probability distributions $[d_1, d_2, \ldots]$, and a sequence of predictions $[\hat{y}_1, \hat{y}_2, \ldots]$. The ground truth output sequence is $[y_1^*, y_2^*, \ldots]$, and the decoder's first input is a start symbol, written y_0^*. Assume we train with teacher forcing to minimize cross-entropy loss, given by the sum of a sequence of losses, one per output timestep, $[\ell_1, \ell_2, \ldots]$. All sequences have finite length. In each answer, list only the quantities from which the given one is computed directly, in one step.

(a) Fill in the blank: The encoder hidden state at time t, b_t, is a function of _____.

(b) Fill in the blank: The decoder hidden state at time t, c_t, is a function of _____.

(c) Fill in the blank: The decoder output probability distribution at time t, d_t, is a function of _____.

(d) Fill in the blank: The loss at time t, ℓ_t, is a function of _____.

CMU *10-418 Machine Learning for Structured Data* **2022** Practice problems 1, Q2.2 · Solutions

55. **Running an RNN language model.** An RNN language model computes its state by the recursion $h_t = f(h_{t-1}, w_t)$, starting from a fixed state h_0 of d numbers. An output matrix V, with one row for each word of the vocabulary and one for `</s>`, gives the next-word probabilities $p(w_t \mid w_1 \ldots w_{t-1}) = \text{softmax}(Vh_{t-1})_{w_t}$. Now answer the following questions in terms of the length n of the text.

(a) What is the runtime of computing the probability $p(w_t \mid w_1 \ldots w_{t-1})$?

(b) What is the runtime of computing the probability $p(w_1 \ldots w_n)$?

(c) Considering the answers to the previous two questions, what does the common claim that "RNN language models are non-parallelizable" mean? How does that limit training on large corpora?

(d) How many computational steps lie between reading w_1 and computing $p(w_t \mid w_1 \dots w_{t-1})$, even if the computation is fully parallelized?

ETH Zurich *Large Language Models* **2024** Week 3 exercises, Q1, parts 1 to 4 · Solutions

56. **The final hidden state.** You build a sentiment analysis system that feeds a sentence into an RNN, and then computes the sentiment class between 0 (very negative) and 4 (very positive), based only on the **final** hidden state of the RNN. You observe that your model predicts very positive sentiment for the following passage:

```
Yesterday turned out to be a terrible day.
I overslept my alarm clock, and to make matters worse,
my dog ate my homework. At least my dog seems happy...
```

Why might the model misclassify the sentiment of this passage?

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Midterm, Q5(b)(ii) · Solutions

57. **Bidirectional RNNs.** A bidirectional RNN runs one RNN over a sentence from left to right and a second RNN from right to left, and at each position joins their two hidden states. In 2–3 concise sentences, describe why it is not appropriate for language modeling.

CMU *10-601 Introduction to Machine Learning* **2025** Study guide 3, Q2.4 · Solutions